Class Overview

In Class:
- Graph Theory for Testers, Continued
- Break (10 minutes)
- Graph Theory for Testers, Continued
- Discrete Math for Testers
- Break (10 minutes)
- Discrete Math for Testers, Continued
- Boundary Value Testing

After Class: Office Hours

Homework:
- 01-28: Syllabus Quiz
- 01-28: Unit Testing Frameworks
- 02-04: Symbolic Execution

No New Homework!

1

Extensions

- Extensions Rules
  - Not available after the due date
  - Not available after the last day of *this* class, not including the exam
  - Must be requested via e-mail
  - You may get early reviews during the extended period after the original due date
  - You may ask for an additional extension before an extended due date

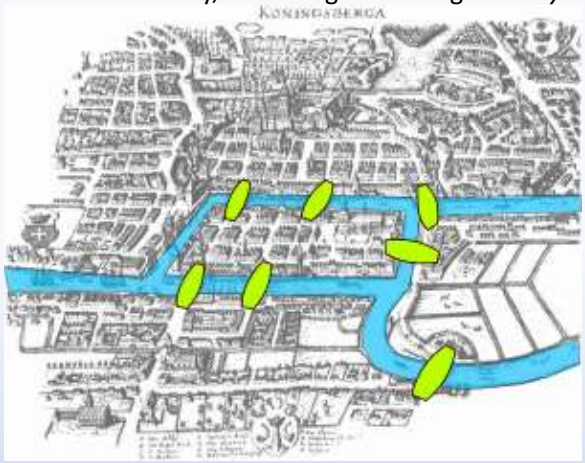
2

CIS 613:
Graph Theory for Testers

3

Bridges of Königsberg Puzzle

- Take a walk around the city, traversing each bridge *exactly once*



https://en.wikipedia.org/wiki/Seven_Bridges_of_Königsberg

4

Bridges of Königsberg Puzzle

- Euler formulated a model that made the problem easier to solve:

Turns out, crossing each bridge only once is impossible.

5

Undirected Graphs

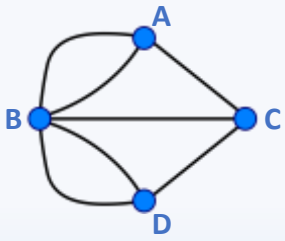
Definition 1: A graph *G = (V, E)* is composed of a finite (and nonempty) set of nodes (*V*) and a set of unordered pairs of nodes (*E*) representing edges.

- When draw, graphs usually show nodes as circles, and edges as lines.
- Leonhard Euler developed graphs to solve the "Bridges of Königsberg" puzzle in 1734.

6

Undirected Graphs

Definition 1: A graph *G = (V, E)* is composed of a finite (and nonempty) set of nodes (*V*) and a set of unordered pairs of nodes (*E*) representing edges.

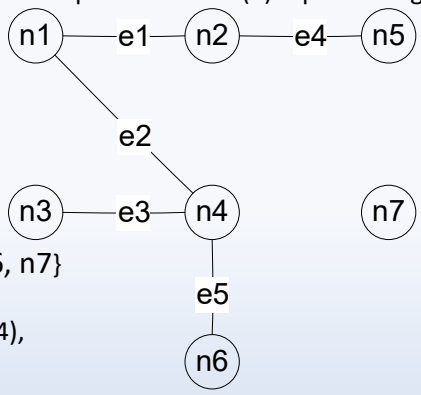


- V = {A, B, C, D}
- E = {(A, B), (A, B), (A, C),
  (B, D), (B, D), (B, C),
  (D, C)}

7

Undirected Graphs: Additional Example

Definition 1: A graph *G = (V, E)* is composed of a finite (and nonempty) set of nodes (*V*) and a set of unordered pairs of nodes (*E*) representing edges.



- V = {n1, n2, n3, n4, n5, n6, n7}
- E = {e1, e2, e3, e4, e5}
  = {(n1, n2), (n1, n4), (n3, n4),
  (n2, n5), (n4, n6)}

8

Degree of a Node

Definition 2: The *degree of a node* in a graph is the number of edges that have that node as an endpoint.

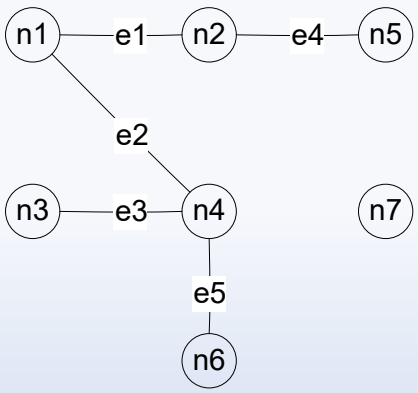
- We write *deg(n)* for the degree of node *n*.
- A type of popularity in the graph. In fact, some folks use graphs to describe
  - Social Interactions,
  - Friendship, and
  - Communication.

9

Degree of a Node

Definition 2: The *degree of a node* in a graph is the number of edges that have that node as an endpoint.

- *deg*(n1) = 2
- *deg*(n2) = 2
- *deg*(n3) = 1
- *deg*(n4) = 3
- *deg*(n5) = 1
- *deg*(n6) = 1
- *deg*(n7) = 0

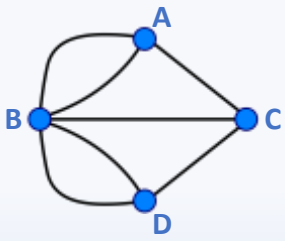


10

Degree of a Node

- What is the degree of node A (i.e., *deg*(A))?

A. *deg*(A) = 0
B. *deg*(A) = 1
C. *deg*(A) = 2
D. *deg*(A) = 3
E. *deg*(A) = 4
F. *deg*(A) = 5

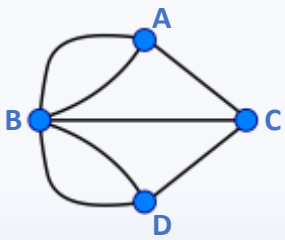


11

Degree of a Node

- Which node, if any, doesn't have a degree of 3?

A. Node A
B. Node B
C. Node C
D. Node D

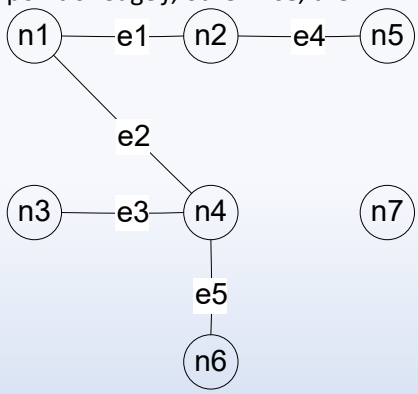


12

Incident Matrix

Definition 3: The *incidence matrix* of a graph $G = (V, E)$ with m nodes and n edges is an m by n matrix, where the element in row i, column j is a 1 if and only if node i is an endpoint of edge j; otherwise, the element is 0.

	*e*1	*e*2	*e*3	*e*4	*e*5
n1	1	1	0	0	0
n2	1	0	0	1	0
n3	0	0	1	0	0
n4	0	1	1	0	1
n5	0	0	0	1	0
n6	0	0	0	0	1
n7	0	0	0	0	0



What is true about the rows in an incident matrix? The columns?

13

Adjacency Matrix of a Graph

Definition 4: The *adjacency matrix* of a graph $G = (V, E)$ with m nodes is an m by m matrix, where the element in row i, column j is a 1 if and only if an edge exists between node i and node j; otherwise, the element is 0.

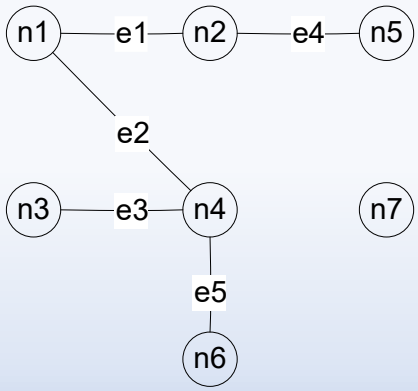
- This kind of matrix can be used to answer questions about a graph, including:
  - Reachability
  - Points affected by a fault
  - Connectedness

14

Adjacency Matrix of a Graph

Definition 4: The *adjacency matrix* of a graph $G = (V, E)$ with m nodes is an m by m matrix, where the element in row i, column j is a 1 if and only if an edge exists between node i and node j; otherwise, the element is 0.

	*n*1	*n*2	*n*3	*n*4	*n*5	*n*6	*n*7
n1	**0**	**1**	**0**	**1**	**0**	**0**	**0**
n2	**1**	**0**	**0**	**0**	**1**	**0**	**0**
n3	**0**	**0**	**0**	**1**	**0**	**0**	**0**
n4	**1**	**0**	**1**	**0**	**0**	**1**	**0**
n5	**0**	**1**	**0**	**0**	**0**	**0**	**0**
n6	**0**	**0**	**0**	**1**	**0**	**0**	**0**
n7	**0**	**0**	**0**	**0**	**0**	**0**	**0**

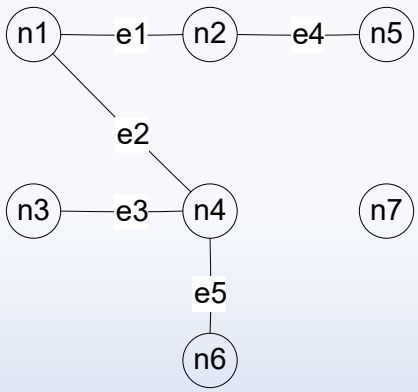


15

Adjacency Matrix of a Graph

Why is this matrix symmetric?

	*n*1	*n*2	*n*3	*n*4	*n*5	*n*6	*n*7
n1	**0**	1	0	1	0	0	0
n2	1	**0**	0	0	1	0	0
n3	0	0	**0**	1	0	0	0
n4	1	0	1	**0**	0	1	0
n5	0	1	0	0	**0**	0	0
n6	0	0	0	1	0	**0**	0
n7	0	0	0	0	0	0	**0**



16

Paths in a Graph

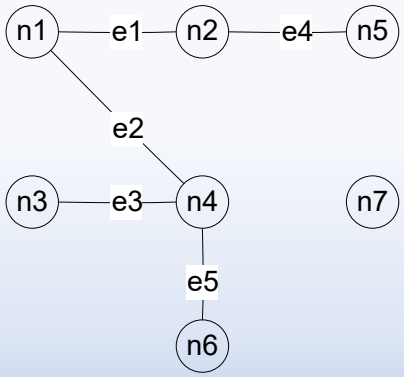
Definition 5: A path is a sequence of edges such that, for any adjacent pair of edges in the sequence, the edges share a common (node) endpoint.

- Paths capture / express connectively
- Paths can be described by
  - A sequence of nodes, or
  - A sequence of edges

17

Example Paths in a Graph

Between	Nodes	Edges
n1 and n5	n1, n2, n5	e1, e2
n6 and n5	n6, n4, n1, n2, n5	e5, e2, e1, e4
n3 and n2	n3, n4, n1, n2	e3, e2, e1



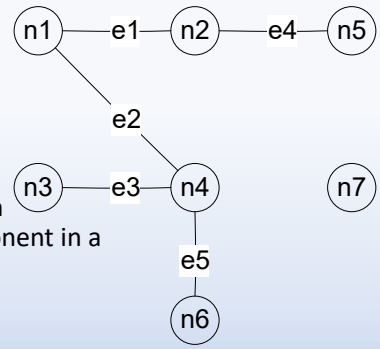
18

Connectedness & Condensation Graphs

Definition 6: Two nodes are *connected* if and only if they are in the same path.

Definition 7: A *component* of a graph is a maximal set of connected nodes.

- C1 = {n1, n2, n3, n4, n5, n6}
- C2 = {n7}



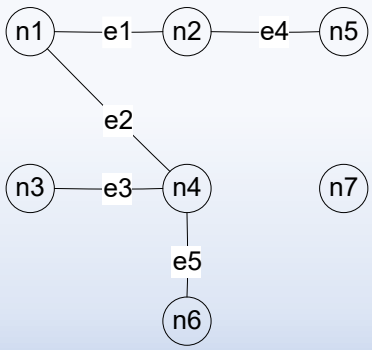
Definition 8: A condensation graph is formed by replacing each component in a graph *G* by a condensing node.

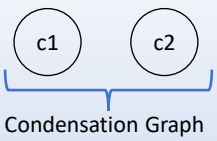
19

Condensation Graphs

- Can a condensation graph have any edges?
  - **Yes**. Of course graphs can have edges.
  - **No**. Of course not, and my reasoning is: _____

- C1 = {n1, n2, n3, n4, n5, n6}
- C2 = {n7}





20

Directed Graphs

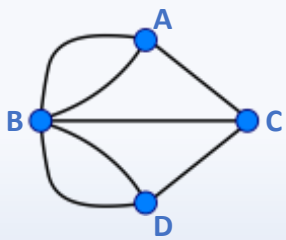
Definition 9: A directed graph (or digraph) $D = (V, E)$ consists of a finite set V of nodes, and a set E of edges, where each edge is an ordered pair of nodes in the set of nodes (i.e., V).

- Note, in this class:
  - $\langle x, y \rangle$ is an ordered pair
  - (x, y) is an unordered pair
- For an edge $\langle a, b \rangle$ in a directed graph, node a is the initial node, and b is the terminal node.

21

Directed Graphs

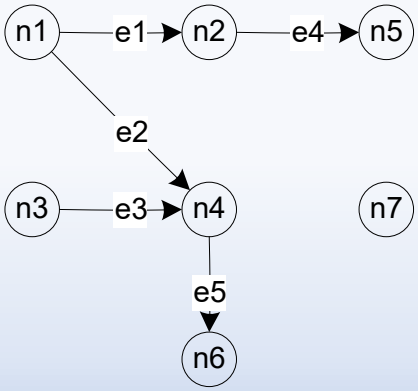
- Does it make sense to convert our bridge map / linear graph into a directed graph?
  - **Yes**, of course.
  - **No**, of course not.



22

Directed Graphs

Definition 9: A directed graph (or digraph) $D = (V, E)$ consists of a finite set V of nodes, and a set E of edges, where each edge is an ordered pair of nodes in the set of nodes (i.e., V).



- V = {n1, n2, n3, n4, n5, n6, n7}
- E = {e1, e2, e3, e4, e5}
  = {<n1, n2>, <n1, n4>, <n3, n4>, <n2, n5>, <n4, n6>}

23

Indegrees and Outdegrees of a Node

Definition 10: The indegree of a node in a directed graph is the number of distinct edges that have the node as a terminal node.

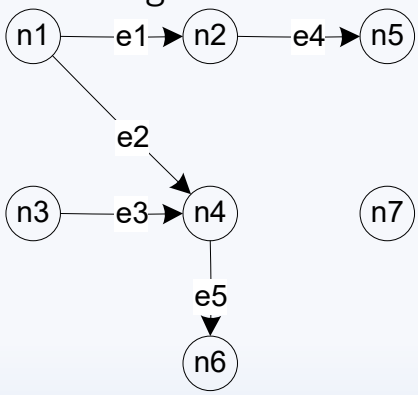
Definition 11: The outdegree of a node in a directed graph is the number of distinct edges that have the node as a start point.

- Note, we write:
  - *indeg*(*n*) for the indegree of node *n,* and
  - *outdeg*(*n*) for the outdegree of node *n*.

24

Indegrees and Outdegrees

indeg(n1) = 0 outdeg(n1) = 2
indeg(n2) = 1 outdeg(n2) = 1
indeg(n3) = 0 outdeg(n3) = 1
indeg(n4) = 2 outdeg(n4) = 1
indeg(n5) = 1 outdeg(n5) = 0
indeg(n6) = 1 outdeg(n6) = 0
indeg(n7) = 0 outdeg(n7) = 0



Definition 12: A node with indegree = 0 is a **source** node.
A node with outdegree = 0 is a **sink** node.
A node with indegree ≠ 0 and outdegree ≠ 0 is a **transfer** node

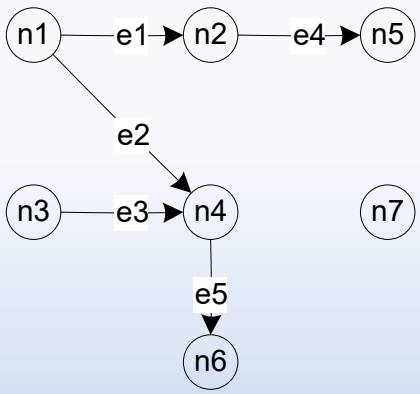
Which nodes are sink, source, and transfer?

25

Adjacency Matrix of a Graph

Definition 13: The *adjacency matrix* of a directed graph *G* = (*V*, *E*) with *m* nodes is an *m* by *m* matrix, where the element in row *i*, column j is a 1 if and only if an edge exists between node *i* and node *j* where node *i* is first and node *j* is second; otherwise, the element is 0.

	*n*1	*n*2	*n*3	*n*4	*n*5	*n*6	*n*7
n1	**0**	**1**	**0**	**1**	**0**	**0**	**0**
n2	**0**	**0**	**0**	**0**	**1**	**0**	**0**
n3	**0**	**0**	**0**	**1**	**0**	**0**	**0**
n4	**0**	**0**	**0**	**0**	**0**	**1**	**0**
n5	**0**	**0**	**0**	**0**	**0**	**0**	**0**
n6	**0**	**0**	**0**	**0**	**0**	**0**	**0**
n7	**0**	**0**	**0**	**0**	**0**	**0**	**0**



26

Adjacency Matrix

- Are adjacency matrices of directed graphs symmetric?

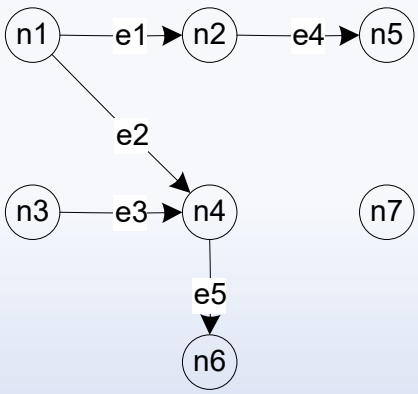
A. Yes, all the time – just like adjacency matrices of undirected graphs.
B. Maybe some of the time. But not all of the time.
C. No, they are never symmetric.

27

In Class Assignment: Adjacency Matrix

Update this graph so that the adjacency matrix is symmetric, **without** removing existing edges.

	*n*1	*n*2	*n*3	*n*4	*n*5	*n*6	*n*7
n1							
n2							
n3							
n4							
n5							
n6							
n7							



28

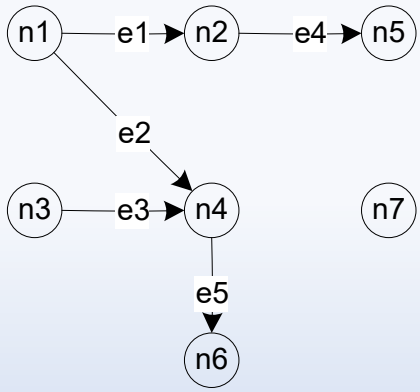
In Class Assignment: Adjacency Matrix

If **rows** of an adjacency matrix for an undirected graph are each node's out degree, what are **columns** of a directed graph's adjacency matrix?

	*n*1	*n*2	*n*3	*n*4	*n*5	*n*6	*n*7
n1	**0**	**1**	**0**	**1**	**0**	**0**	**0**
n2	**0**	**0**	**0**	**0**	**1**	**0**	**0**
n3	**0**	**0**	**0**	**1**	**0**	**0**	**0**
n4	**0**	**0**	**0**	**0**	**0**	**1**	**0**
n5	**0**	**0**	**0**	**0**	**0**	**0**	**0**
n6	**0**	**0**	**0**	**0**	**0**	**0**	**0**
n7	**0**	**0**	**0**	**0**	**0**	**0**	**0**

outdegree

indegree



29

Directed Graphs

Definition 14: A (directed) path is a sequence of edges such that, for any adjacent pair of edges in the sequence, the terminal node of the first edge is the initial node of the second edge.

Additionally:

- A **cycle** is a path in which some node is both the initial and the final node in the path.
- A **chain** is a sequence of nodes in which every interior node has indegree = outdegree = 1
- A **semipath** is a sequence of edges such that at least one node is either the initial node or the terminal node of two adjacent edges in the sequence.

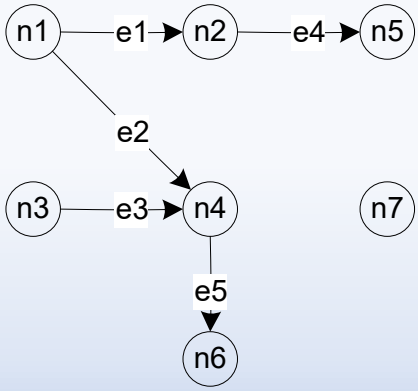
30

Directed Graphs: Semipaths

A **semipath** is a sequence of edges such that at least one node is either the initial node or the terminal node of two adjacent edges in the sequence.

(Some) Semipaths:

- n1 and n3 (via n4)
- n2 and n4 (via n1)
- n5 and n6 (via n1)

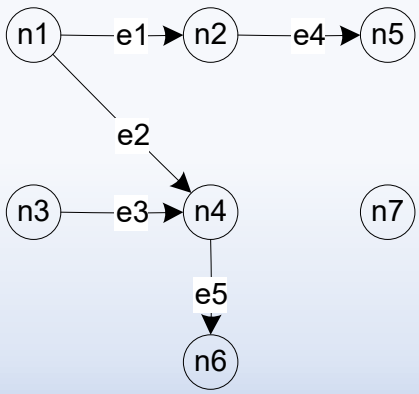


31

Reachability Matrix

- Computing a reachability matrix based on an adjacency matrix *(A)*:
  - Reachable in **one step** = Adjacency Matrix (i.e., *A*)

	*n*1	*n*2	*n*3	*n*4	*n*5	*n*6	*n*7
n1	**0**	**1**	**0**	**1**	**0**	**0**	**0**
n2	**0**	**0**	**0**	**0**	**1**	**0**	**0**
n3	**0**	**0**	**0**	**1**	**0**	**0**	**0**
n4	**0**	**0**	**0**	**0**	**0**	**1**	**0**
n5	**0**	**0**	**0**	**0**	**0**	**0**	**0**
n6	**0**	**0**	**0**	**0**	**0**	**0**	**0**
n7	**0**	**0**	**0**	**0**	**0**	**0**	**0**

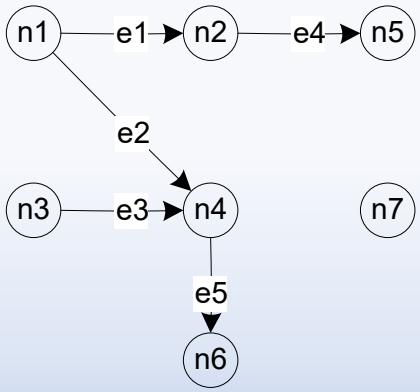


32

Reachability Matrix

- Computing a reachability matrix based on an adjacency matrix *(A)*:
  - Reachable in **two steps** = Adjacency Matrix (i.e., A^2)

	*n*1	*n*2	*n*3	*n*4	*n*5	*n*6	*n*7
n1	**0**	**0**	**0**	**0**	**1**	**1**	**0**
n2	**0**	**0**	**0**	**0**	**0**	**0**	**0**
n3	**0**	**0**	**0**	**0**	**0**	**1**	**0**
n4	**0**	**0**	**0**	**0**	**0**	**0**	**0**
n5	**0**	**0**	**0**	**0**	**0**	**0**	**0**
n6	**0**	**0**	**0**	**0**	**0**	**0**	**0**
n7	**0**	**0**	**0**	**0**	**0**	**0**	**0**



33

Where is all this going?

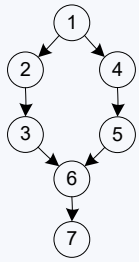
Program Graphs!

34

Program Graphs

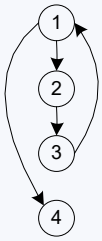
If-Then-Else

```
1  If <condition>
2     Then
3        <then statements>
4     Else
5        <else statements>
6  End If
7  <next statement>
```



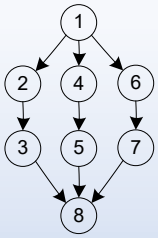
Pre-test Loop

```
1  While <condition>
2     <repeated body>
3  End While
4  <next statement>
```



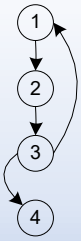
Case/Switch

```
1  Case  n Of 3
2     n=1:
3        <case 1  statements>
4     n=2:
5        <case 2  statements>
6     n=3:
7        <case 3  statements>
8  End Case
```



Post-test Loop

```
1  Do
2     <repeated body>
3  Until <condition>
4  <next statement>
```

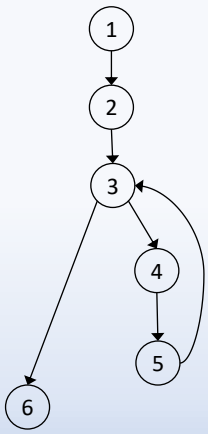


35

Basis Path Testing

- **Step 1**: Use the design or code as a foundation to draw a corresponding flow graph.

```
def sum_numbers(n):

1 → i = 1
2 → total = 0

3 → while i <= n:
4 →    total += 1
5 →    i += 1

6 → return total
```



36

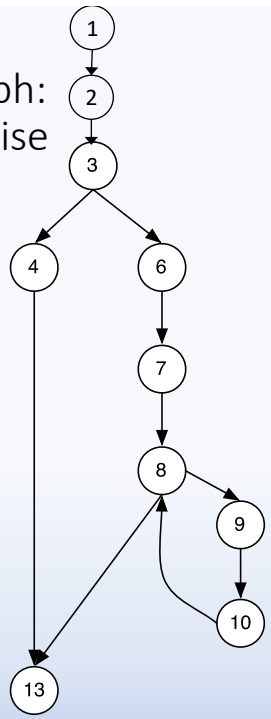
Program Graph: In Class Exercise

```
1.  int factorial(int n) {
2.      int i, fact;
3.      if(n < 0) {
4.          fact = 0;
5.      } else {
6.          i = 1;
7.          fact = 1;
8.          while(i <= n) {
9.              fact = fact * i;
10.             i = i + 1;
11.         }
12.     }
13.     return fact;
14. }
```

37

Program Graph: In Class Exercise

```
1.  int factorial(int n) {
2.      int i, fact;
3.      if(n < 0) {
4.          fact = 0;
5.      } else {
6.          i = 1;
7.          fact = 1;
8.          while(i <= n) {
9.              fact = fact * i;
10.             i = i + 1;
11.         }
12.     }
13.     return fact;
14. }
```



38

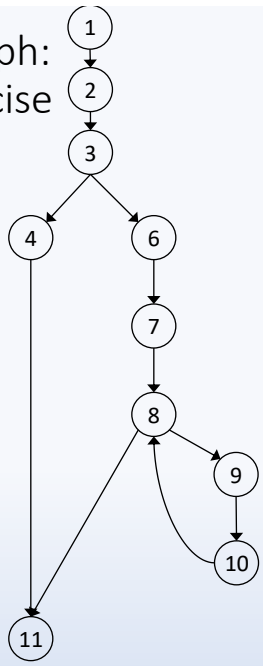
Program Graph: In Class Exercise

```
1.  def factorial(n):

2.      fact = None
3.      if n < 0:
4.          fact = 0
5.      else:
6.          i = 1
7.          fact = 1
8.          while i <= n:
9.              fact = fact * i
10.             i = i + 1

11.     return fact
```

39

Program Graph: In Class Exercise

```
1.  def factorial(n):

2.      fact = None
3.      if n < 0:
4.          fact = 0
5.      else:
6.          i = 1
7.          fact = 1
8.          while i <= n:
9.              fact = fact * i
10.             i = i + 1

11.     return fact
```



40

Program Graphs: Extra In Class Exercise

```
-  // Method returns the greatest common divisor of a and b
-
-  private static int gcd (int a, int b) {
-
1     if(a < 0) {
2        a = Integer.MIN_VALUE;
3     } else if(b < 0) {
4        a = Integer.MIN_VALUE;
5     } else {
6        while (b != 0) {
7           int temp = b;
8           b = a % b;
9           a = temp;
-        }
-     }
-
10    return a;
-  }
```

41

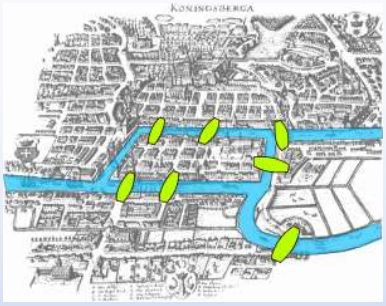
Basis Path Testing: In Class Exercise

```
1.  def gcd(a, b):

2.     if a < 0:
3.        a = None
4.     elif b < 0:
5.        a = None
6.     else:
7.        while b != 0:
8.           temp = b
9.           b = a % b
10.          a = temp

11.    return a
```

42

Bridges of Königsberg Puzzle

- Take a walk around the city, traversing each bridge *exactly once*
  - *Impossible!*
  - *Euler observed the following was needed in the graph:*
    - Must be **Connected**, and
    - Have **Zero or Two nodes of Odd Degree**

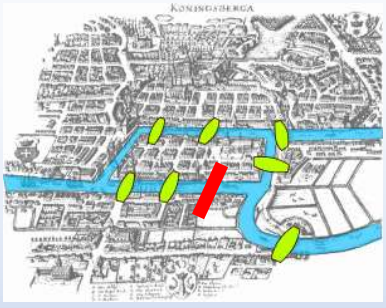


https://en.wikipedia.org/wiki/Seven_Bridges_of_Königsberg

43

Bridges of Königsberg Puzzle

- Take a walk around the city, traversing each bridge *exactly once*
  - *Impossible!*
  - *Euler observed the following was needed in the graph:*
    - Must be **Connected**, and
    - Have **Zero or Two nodes of Odd Degree**



https://en.wikipedia.org/wiki/Seven_Bridges_of_Königsberg

44